E

Everything School

MANAGEMENT SYSTEM

Comprehensive School Management Platform

Multi-tenant SaaS · Django + React · Enterprise-Grade

DJANGO 5.2 | REACT 19 | POSTGRESQL | TYPESCRIPT

MULTI-TENANT | WEASYPRINT PDF | REST API | JWT AUTH

24+	4	51+
DJANGO APPS	EDUCATION LEVELS	API SERVICES

Table of Contents

01 Project Overview 3

02 Tech Stack 4

03 Features 5

04 Architecture Explanation 7

05 Authentication Flow 9

06 Setup Guide 10

07 Deployment Stack 12

08 Challenges Solved 14

01 Project Overview

What it is, who it serves, and why it was built

Everything School Management System v2 is an enterprise-grade, multi-tenant SaaS platform designed to digitise and streamline the complete operational lifecycle of K-12 schools — from nursery through senior secondary. Built with a Django REST backend and a React/TypeScript frontend, the platform serves four distinct user roles (Admin, Teacher, Student, Parent) and supports multiple education levels each with their own curriculum, grading, and reporting requirements.

CORE MISSION

Provide schools with a single, unified platform to manage academics, attendance, results, fees, communication, and reporting — eliminating the need for multiple disconnected tools while maintaining data isolation between schools through a robust multi-tenancy architecture.

Who Uses It

ADMINISTRATORS

- Full school configuration and management
- User management and permissions
- Reports, approvals, and publishing
- Fee structures and financial overview

TEACHERS

- Lesson planning and delivery tracking
- Attendance marking per class/subject
- Component-based score entry
- Digital remark and signature submission

STUDENTS

- View published term and session results
- Access timetable and schedules
- Check exam information
- Token-gated result access

PARENTS

- Monitor ward's academic progress
- Receive fee notifications
- Communicate with teachers
- Access school announcements

Education Levels Supported

LEVEL	GRADES	SPECIAL FEATURES
Nursery	Pre-Nursery, Nursery 1–2	Developmental tracking (physical, health, cleanliness, conduct), mark-based scoring
Primary	Primary 1–6	Component-based CA + Exam scoring, class position ranking
Junior Secondary	JSS 1–3	Subject position ranking, term/session report hierarchy

Senior Secondary	SSS 1–3	Stream-based curriculum (Science, Arts, Commerce, Technical), elective subjects, stream-specific assessment

02 Tech Stack

Every technology powering the platform

BACKEND CORE

- Django 5.2.1 — web framework
- Django REST Framework 3.16 — API layer
- PostgreSQL — primary database (Supabase hosted)
- Gunicorn — WSGI production server
- Celery + Redis — async task queue

FRONTEND CORE

- React 19.1.0 — UI library
- TypeScript 5.8.3 — type safety
- Vite 6.3.5 — build tool and dev server
- React Router v7 — client-side routing
- Tailwind CSS 3.4 — utility-first styling

AUTHENTICATION & SECURITY

- SimpleJWT — JSON Web Tokens with rotation
- django-allauth — social login provider
- Google OAuth 2.0 — social sign-in
- dj-rest-auth — registration / verification
- httpOnly cookies + CSRF protection

FILE & STORAGE

- Cloudinary — image/file hosting & CDN
- django-cloudinary-storage — Django integration
- Supabase Storage — document uploads
- WhiteNoise — static file serving

DOCUMENT GENERATION

- WeasyPrint — server-side PDF reports
- jsPDF + html2canvas — client PDF export
- python-docx — Word document generation
- PyPDF2 / pdfplumber — PDF parsing
- TipTap — rich text editing

PAYMENTS & COMMS

- Stripe 8.11 — card payment processing
- Paystack 1.5 — local payment gateway
- Twilio 9.8 — SMS notifications
- django-filter — advanced API filtering
- Recharts 3.1 — data visualisation

INFRASTRUCTURE

Hosting: Render (backend + frontend static) · **Database:** Supabase PostgreSQL with SSL · **CDN:** Cloudflare (SaaS custom domains) · **CI/CD:** Git-triggered auto-deploy on Render

03 Features

Full capability map across all modules

Academic Management

MODULE	CAPABILITIES
Curriculum	Academic sessions, terms, grade levels, stream-based subject assignments, elective configuration
Timetable	Automated scheduling, period management, teacher-subject-class mapping, conflict detection
Lessons	Lesson planning with notes, resources, feedback, attendance; 24-hour data retention policy; downloadable reports
Attendance	Daily per-student attendance (Present/Absent/Late/Excused); attendance rate calculation on term reports
Examinations	Exam creation with Objectives, Theory, Practical, Custom sections; templates; scheduling; bulk document import; approval workflow

Result Management System

MULTI-LEVEL RESULT HIERARCHY

Each education level has its own result pipeline: **Subject Results** (component scores) → **Term Report** (aggregated metrics + class position) → **Session Report** (annual summary across all terms)

- Configurable assessment components (CA 1, CA 2, Project, Exam) per school
- Admin-defined grading systems with custom grade ranges and remarks
- SQL RANK()-based class and subject position calculation
- Digital teacher signatures and remarks on term reports
- Server-side PDF generation (WeasyPrint) with school logo, student photo, section, grading scale
- Next-term-begins date from Academic Year settings in PDF
- Attendance integration — school days from `weeks_per_term × 5` setting
- Token-gated result access for students

User & School Management

Students

- Registration with education level and stream
- Bulk upload via CSV/Excel
- Class promotion with configurable rules
- Profile picture on term report PDF
- Parent linking

Teachers

- Subject specialisation and level assignment
- Subject-classroom assignment with periods/week
- Assignment request workflow
- Digital signature management

Multi-Tenancy

- Each school is an isolated tenant
- Subdomain routing (slug.domain.com)
- Custom domain support via Cloudflare
- Per-tenant settings (branding, config)

Communication

- Internal messaging between roles
- School announcements and notices
- SMS via Twilio integration
- Event management and sharing

Financial Management

- Flexible fee structures with installment plans
- Dual payment gateway: **Stripe** (international) and **Paystack** (local)
- Invoice generation, tracking, and overdue notifications
- Fee discount management per student

04 Architecture Explanation

How the system is structured and how components interact

High-Level Architecture

CLIENT	Browser (React 19 + TypeScript) · Vite Dev Server / CDN Static · Tailwind CSS
PROXY	Vite Proxy (/api → :8000) · Cloudflare WAF + CDN · X-Forwarded-Host header
API	Django REST Framework · Gunicorn WSGI · TenantMiddleware · JWT CookieAuth
SERVICES	Celery + Redis · WeasyPrint PDF · Cloudinary Storage · Stripe / Paystack
DATA	Supabase PostgreSQL · Redis Cache · Cloudflare KV

Multi-Tenant Request Flow

Request slug.school.com/api/... → **TenantMiddleware** Resolve tenant from subdomain → **JWT Auth** Cookie + CSRF validate → **ViewSet** Filter by tenant FK → **Response** Tenant-scoped data

Backend App Structure

DJANGO APP	RESPONSIBILITY	KEY MODELS
`academics`	Academic calendar management	AcademicSession, Term, EducationLevel, GradeLevel
`classroom`	Physical class/section management	Classroom, Section, Stream, ClassroomTeacherAssignment
`result`	Grading and report generation	28 models (4 result levels × 3 types + supporting)

`exam`	Exam lifecycle management	Exam, ExamQuestion, ExamSchedule, ExamTemplate
`tenants`	Multi-tenancy and school config	Tenant, TenantSettings, CustomDomain
`lesson`	Lesson tracking with retention	Lesson, LessonAttendance
`fee`	Financial management	FeeStructure, Invoice, Payment, Installment
`attendance`	Student attendance records	Attendance (date, student, status)

Frontend Architecture

SRC/ — FRONTEND DIRECTORY STRUCTURE

```
src/
├── App.tsx                    # Root with lazy-loaded route tree
├── components/
│   └── dashboards/
│       ├── admin/             # 150+ admin dashboard components
│       ├── teacher/           # 25+ teacher components
│       └── student/           # 16+ student components
├── services/                  # 51 typed API service files
│   ├── ResultService.ts       # Comprehensive result CRUD + PDF
│   ├── AuthService.ts         # JWT token management
│   └── ...                    # One service per domain
├── contexts/                  # React context providers (auth, tenant)
├── hooks/                     # Custom React hooks
├── pages/                     # Top-level route pages
├── routes/                    # Protected route wrappers (RBAC)
└── types/                     # TypeScript interfaces
```

05 Authentication Flow

JWT tokens, cookies, CSRF, and social login

Standard Login Flow (JWT + Cookie)

1. Credentials Submitted
POST /api/auth/login/ → { username, password, tenant_slug }

↓

2. JWT Issued
access_token (httpOnly cookie) + refresh_token (httpOnly cookie) + csrftoken

↓

3. Authenticated Requests
Cookie sent automatically + X-CSRFToken header on mutating requests

↓

4. Token Refresh
Automatic POST /api/auth/refresh/ when access_token expires (detected by 401 response)

Google OAuth Flow

User clicks "Sign in with Google" → **Google consent** OAuth popup → **ID token sent** POST /api/auth/google/ → **django-allauth** Verify + create/link account → **JWT cookies set** Same as standard flow

Role-Based Access Control (RBAC)

ROLE	IDENTIFIER	ACCESSIBLE ROUTES
Superadmin / Admin	`is_staff=True` or role=admin	All admin dashboard routes + full API access
Teacher	UserRole = teacher	Teacher dashboard, own classes/subjects, own lessons
Student	UserRole = student	Student portal, own results (token-gated), own schedule

Parent	UserRole = parent	Parent portal, linked ward's progress, messaging

SECURITY HEADERS (PRODUCTION)

HTTPS-only · HSTS with subdomains · X-Frame-Options: DENY · XSS filter · CSRF_COOKIE_SECURE · SESSION_COOKIE_SECURE · SECURE_CONTENT_TYPE_NOSNIFF

06 Setup Guide

Get the system running locally from scratch

PREREQUISITES

Python 3.11+ · Node.js 18+ · PostgreSQL 14+ · Redis 7+ · Git

Backend Setup

1 Clone and create virtual environment

```
git clone https://github.com/your-org/everything-school-management-v2.git
cd everything-school-management-v2/backend
python -m venv venv
# Windows:  venv\\Scripts\\activate
# macOS/Linux: source venv/bin/activate
pip install -r requirements.txt
```

2 Configure environment variables

```
BACKEND/.ENV.DEV

DEBUG=True
ENV=development
LOCAL_DATABASE_URL=postgres://user:password@localhost:5432/school_db
DJANGO_SECRET_KEY=your-secret-key-here
FRONTEND_URL=http://localhost:5173
ALLOWED_HOSTS=localhost,127.0.0.1

# Optional integrations
CLOUDINARY_CLOUD_NAME=your_cloud
CLOUDINARY_API_KEY=your_key
CLOUDINARY_API_SECRET=your_secret
GOOGLE_CLIENT_ID=your_google_client_id
STRIPE_SECRET_KEY=sk_test_ ...
```

3 Run database migrations and start server

```
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Frontend Setup

4 Install dependencies and configure

```
FRONTEND/.ENV

VITE_API_URL=/api
VITE_FRONTEND_URL=http://localhost:5173
VITE_GOOGLE_CLIENT_ID=your_google_client_id.apps.googleusercontent.com
```

```
cd ../frontend
npm install
npm run dev       # Starts on http://localhost:5173
```

5 Create your first school tenant

- Visit `http://localhost:5173/register-school`
- Complete the onboarding wizard (school name, level configuration, admin user)
- Access your school at `http://your-slug.localhost:5173`

Optional: Run with Docker

```
DOCKER-COMPOSE.YML

docker-compose up --build
# Backend available at http://localhost:8000
# Configure PROD_DATABASE_URL in .env for production DB
```

SEED DATA

Run `python manage.py setup_result_data` to seed grading systems and assessment components.
Run `python manage.py setup_academic_subjects` to seed standard subjects by level.

07 Deployment Stack

Production infrastructure and CI/CD pipeline

Production Infrastructure

DNS	Cloudflare — WAF, DDoS protection, SSL termination; SaaS custom hostname routing
FRONTEND	Render Static Site; Vite production build (dist/); Auto-deploy on git push
BACKEND	Render Web Service; Gunicorn 2 workers; 120s timeout
DATABASE	Supabase PostgreSQL; SSL required (verify-full); Connection pooling
STORAGE	Cloudinary (images, PDFs); Supabase Storage (documents); WhiteNoise (static assets)

CI/CD Pipeline

Developer pushes to main branch

↓

Render detects commit → triggers build

build.sh: pip install → collectstatic → migrate

↓

New Gunicorn instance replaces old one (zero-downtime)

Build Script

```
BACKEND/BUILD.SH
```

```
#!/usr/bin/env bash
set -o errexit

pip install -r requirements.txt
python manage.py collectstatic --no-input
python manage.py migrate
```

Required Production Environment Variables

VARIABLE	DESCRIPTION	REQUIRED
`PROD_DATABASE_URL`	Supabase PostgreSQL connection string	✓ Yes
`DJANGO_SECRET_KEY`	50+ character random secret	✓ Yes
`FRONTEND_URL`	Production frontend URL for CORS	✓ Yes
`ENV`	Set to `production`	✓ Yes
`CLOUDFLARE_API_TOKEN`	For custom domain management	Custom domains
`CLOUDINARY_*`	Cloud name, API key and secret	File uploads
`STRIPE_SECRET_KEY`	Stripe payment processing	Payments
`PAYSTACK_PUBLIC_KEY`	Paystack payment gateway	Payments

08 Challenges Solved

Complex technical problems tackled during development

1 · PostgreSQL Window Function + SELECT FOR UPDATE Conflict

PROBLEM PostgreSQL forbids combining `FOR UPDATE` (row locking) with `RANK()` window functions in the same query. The class position ranking system used both simultaneously, causing NotSupportedError on every position recalculation.

SOLUTION Separated the concerns: use `select_for_update()` on the plain queryset to acquire row locks, then run the window annotation on a separate `cls.objects.filter(pk__in=pks)` queryset without the lock. Both operations run inside a single `transaction.atomic()` block to maintain consistency.

2 · Education Level Code Mismatch Silently Zeroed All Positions

PROBLEM The `bulk_recalculate_positions` method filtered by `level_type="JUNIOR_SECONDARY"` but the database stores `"JSS"` for Junior Secondary and `"SSS"` for Senior Secondary. The queryset returned empty silently — no error, but no positions were ever calculated. The success toast fired regardless because the API counted classes found, not positions set.

SOLUTION Removed the redundant `education_level` filter entirely (using `**_` to absorb old callers). The combination of `exam_session` + `student_class` already uniquely scopes the ranking. Added `bulk_calculate_metrics()` before ranking to ensure `average_score` is always fresh.

3 · Windows Temp File Lock Breaking WeasyPrint PDF Generation

PROBLEM On Windows, `NamedTemporaryFile(delete=True)` opens a file with `FILE_FLAG_DELETE_ON_CLOSE`, preventing any other handle from writing to it. WeasyPrint's `write_pdf(target=tmp.name)` tried to open the same file by name and received Permission denied (errno 13).

SOLUTION Eliminated the temp file entirely. `WeasyHTML.write_pdf()` called without a `target` argument returns the PDF as a `bytes` object in memory, which is streamed directly into an `HttpResponse`. Simpler, faster, and fully cross-platform.

4 · Multi-Tenant Result Hierarchy with Education-Level-Specific Models

PROBLEM Four education levels each require different grading logic, different assessment components, different report layouts, and different position-calculation strategies. A single generic model would be too complex; four unrelated models would mean duplicated business logic.

SOLUTION Introduced abstract base classes (`BaseResult`, `BaseTermReport`, `TermReportFields`) containing shared logic. Each level inherits and overrides only what differs. The report generator factory pattern maps education level → generator class, keeping PDF rendering consistent while allowing level-specific context building.

5 · Subject Results Approved Before Metrics Calculated = Zero Averages

PROBLEM When subject results were approved, the term report's `average_score` was never recalculated. `calculate_metrics()` only aggregates APPROVED results, so if it ran before approval it found nothing. Term reports showed 0.0 average and 0.0 total score in PDFs.

SOLUTION Added `calculate_metrics()` / `bulk_calculate_metrics()` to all approve and bulk-approve endpoints, after subject results are approved. Also added a defensive call inside `generate_term_report()` — if `average_score == 0` at PDF time, recalculate on the fly before rendering.

6 · Attendance Status Code Mismatch

PROBLEM The Attendance model stores status as a single character code (`'P'` for Present) but the report generator filtered with `status="PRESENT"` (full string), returning zero attendance for every student despite records existing in the database.

SOLUTION Diagnosed via Django shell: confirmed `Attendance._meta.get_field('status').choices` shows `[('P', 'Present'), ('A', 'Absent') ... ]`. Fixed the filter to use the actual stored value `status="P"`. Times-present now correctly counts from attendance records within the term's date range.

7 · URL Constructor Crashes on Relative API Paths

PROBLEM The PDF download function used `new URL(`${API_BASE_URL}/results/ ... `)`. In development, `VITE_API_URL=/api` (a relative path), causing "Failed to construct 'URL': Invalid URL" — the browser's `URL` constructor requires an absolute URL when called with one argument.

SOLUTION Replaced `new URL()` + `.searchParams.set()` with `new URLSearchParams({ ... })` appended as a query string to a template literal. `fetch('/api/ ... ')` resolves relative paths against the current origin automatically, working in both development (Vite proxy) and production.

E

Everything School Management

Version 2.0 · May 2026

Built with Django 5.2, React 19, PostgreSQL, and WeasyPrint.
Multi-tenant · Role-based · Enterprise-Grade